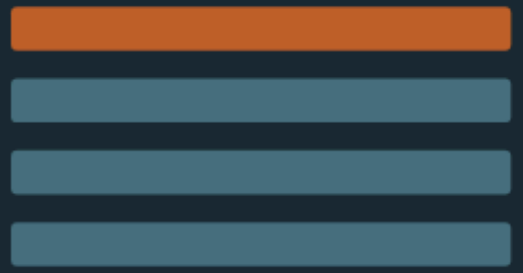
MAESTRÍA · ESPACIO ACADÉMICO

Gestión de Configuración y Mantenimiento de Software



Sesión 1 — Unidad 1 · Evolución del software y mantenimiento

Diagnosticar un sistema a partir de su propia historia.

César Augusto Vega Fernández

Encuentro sincrónico · Leyes de Lehman · ISO/IEC/IEEE 14764 · Modelo por etapas de Rajlich y Bennett

EL RETO DE LA UNIDAD 1 · ACTIVIDADES EN LA PLATAFORMA

Un diagnóstico con el que un comité pueda decidir

«Nuestra organización depende de un componente de código abierto y tenemos que decidir si seguimos apostando por él durante los próximos cinco años.

No queremos opiniones sobre si es popular. Queremos saber cómo ha evolucionado, en qué se le va el esfuerzo de mantenimiento, en qué etapa de su vida está y qué deuda arrastra.

Con eso decidimos.»

Comité de arquitectura de la organización cliente

Actividad 1.1 · Presentación del reto

· Individual · formativa. Reflexión de 1 a 2 páginas: qué tipo de mantenimiento consume más esfuerzo en un sistema que usted conoce, y cómo lo demostraría. Propuesta de proyecto con el F-01 en borrador.

Actividad 1.2 · Diagnóstico de evolución y mantenimiento

· Grupal · peso y horas: por completar

· Métricas por versión · clasificación de 40 cambios

· Contraste con las leyes de Lehman · etapa del ciclo de vida

· 5 deudas técnicas · recomendación · PDF de máximo 15 páginas, APA 7

La evidencia sale del repositorio, del registro de cambios y de los issues. Una afirmación sin dato que la respalde no entra en el informe.

Los F-nn son los formatos del curso, las plantillas que preparan cada sección del informe. F-01 a F-07 son los de la Unidad 1. No se suben a la plataforma.

CÓMO VAMOS A USAR LA SESIÓN

Primero qué significa, después ejemplos, después ustedes

PRIMERO

Conceptos

Por qué el software que se usa tiene que cambiar, las leyes de Lehman, los tipos de mantenimiento, las etapas de vida y la deuda técnica.

DESPUÉS

Ejemplos

Los formatos F-01 a F-06 llenos con la historia real de Flask, un framework web con quince años de versiones.

Y EL RESTO DE LA SESIÓN

El hacer

El taller en salas: elegir el proyecto, clonarlo y correr la herramienta de métricas sobre él.

El bloque que no se recorta es el tercero. Si los conceptos se alargan, se acortan los ejemplos: están completos en el recorrido resuelto del repositorio.

BLOQUE 1 · POR QUÉ CAMBIA EL SOFTWARE

Tres tipos de programa según Lehman

S · especificado

Resuelve un problema definido formalmente.

Es correcto o no lo es respecto de su especificación.

Ejemplo: una función que ordena una lista.

P · problema

Resuelve un problema real con una aproximación aceptable.

Se juzga por la utilidad de la solución.

Ejemplo: un programa que juega ajedrez.

E · embebido

Opera en el mundo real y forma parte de él.

Su uso cambia el entorno, y el entorno cambiado exige cambiar el programa.

Ejemplo: un sistema bancario, un framework web.

Las leyes de la evolución se formularon para los programas tipo E. Casi todo el software que una organización mantiene es de ese tipo.

Lehman, M. M. (1980). Programs, life cycles, and laws of software evolution. Proceedings of the IEEE, 68(9), 1060–1076.

BLOQUE 1 · LEYES DE LA EVOLUCIÓN · HERRAMIENTA EN VIVO

Las ocho leyes de Lehman

Ley	Qué dice	Qué dato la mostraría
I · Cambio continuo	Un sistema E que no se adapta se vuelve progresivamente menos satisfactorio	Cambios adaptativos durante toda la vida
II · Complejidad creciente	La complejidad crece salvo que se trabaje para reducirla	Trabajo preventivo, retiro de código, o indicadores de complejidad al alza
III · Autorregulación	La evolución se autorregula: sus medidas tienen tendencias estables	Tendencias reconocibles en tamaño y actividad
IV · Estabilidad organizacional	La tasa media de trabajo efectivo es invariante	Commits por mes estables aunque cambie el equipo
V · Familiaridad	El contenido incremental de cada versión se mantiene acotado	Después de una versión grande, una pequeña
VI · Crecimiento continuo	La funcionalidad debe crecer para mantener la satisfacción	Tamaño creciente versión tras versión
VII · Calidad decreciente	La calidad se percibe en declive si no se mantiene y adapta	Defectos reportados por versión
VIII · Retroalimentación	La evolución es un sistema de retroalimentación de varios niveles	Decisiones que responden a usuarios, equipo y entorno

Formulación de Lehman et al. (1997). En el aula se recorre el interactivo de las leyes: el control de trabajo preventivo muestra la ley II.

Cuatro tipos de mantenimiento

	Corrección	Mejora
Reactivo	**Correctivo** · repara una falla que ya se manifestó	**Adaptativo** · responde a un cambio del entorno: lenguaje, dependencias, plataforma, normas
Proactivo	**Preventivo** · corrige fallas latentes o reduce la dificultad de cambios futuros	**Perfectivo** · agrega o mejora funcionalidad, rendimiento o usabilidad

De dónde sale

Swanson (1976) propuso correctivo, adaptativo y perfectivo.

La norma ISO/IEC/IEEE 14764 agrega el preventivo y los agrupa en corrección y mejora.

SWEBOK presenta esta misma rejilla.

El tipo se decide leyendo el cambio. El mensaje del commit orienta, pero «fix» aparece en correcciones de documentación y en funcionalidades nuevas.

En el aula se usa el clasificador interactivo: trece cambios reales de Flask para clasificar antes de ver la respuesta.

BLOQUE 1 · EN QUÉ MOMENTO DE SU VIDA ESTÁ

El modelo por etapas de Rajlich y Bennett

Desarrollo inicial	Primera versión funcional. Se forman la arquitectura y el conocimiento del equipo.
Evolución	Funcionalidad nueva frecuente. Los cambios de estructura todavía son posibles porque el equipo entiende la arquitectura.
Servicio	Correcciones y adaptaciones pequeñas. Los cambios de estructura se evitan porque se volvieron costosos o riesgosos.
Retiro gradual	Solo correcciones críticas. Se anuncia reemplazo o fin de soporte.
Cierre	El sistema se apaga y los usuarios migran.

Un sistema pasa de evolución a servicio cuando pierde la integridad de su arquitectura o el conocimiento de quienes la entendían. Ese paso rara vez tiene marcha atrás.

Rajlich, V. T., & Bennett, K. H. (2000). A staged model for the software life cycle. Computer, 33(7), 66–71.

BLOQUE 1 · LO QUE SE ACUMULA

Deuda técnica y envejecimiento

	Imprudente	Prudente
Deliberada	«No hay tiempo para diseñar»	«Hay que publicar ya y asumimos las consecuencias»
Inadvertida	«¿Qué es una arquitectura por capas?»	«Ahora sabemos cómo debimos hacerlo»

Los términos

Interés: lo que cuesta cada cambio mientras la deuda sigue.

Pago: lo que cuesta quitarla.

Cunningham (1992) acuñó la metáfora; Fowler (2009) propuso este cuadrante.

El envejecimiento según Parnas (1994)

El software envejece por dos causas: no se adapta a un entorno que cambia, y los cambios que sí se le hacen los hace gente que no entiende el diseño original. La segunda causa es deuda inadvertida.

BLOQUE 1 · LA CADENA QUE SOSTIENE EL INFORME

De la historia del repositorio a una recomendación

Historia
Etiquetas, commits, registro de cambios, issues

›

Métricas
F-02 · tamaño, crecimiento y actividad por versión

›

Clasificación
F-03 · qué tipo de mantenimiento consume el esfuerzo

›

Leyes y etapa
F-04 y F-05 · qué se observa y en qué momento está

›

Deuda y decisión
F-06 · qué arrastra y qué conviene hacer

Cada eslabón se apoya en el anterior. Una recomendación que no se puede rastrear hasta un dato del repositorio no sostiene la decisión del comité.

BLOQUE 2 · EJEMPLO GUIADO 1 · F-01

Decidir qué se mide antes de medir

flask.py 0.1 a 0.4	Todo el framework en un solo archivo.
flask/ 0.5 a 1.0	Paquete con varios módulos. Hasta la 0.10 las pruebas viven dentro, en **flask/testsuite/**.
src/flask/ desde 1.1.0	El paquete pasa a la carpeta src/. Las pruebas, la documentación y los ejemplos están fuera.

Un error de medición

Sin acotar las rutas, la versión 0.10 aparece con **59 archivos**.

Midiendo solo el paquete tiene **20**.

La diferencia son pruebas y setup.py: cambia cualquier conclusión sobre el crecimiento.

```
python herramientas/metricas_evolucion.py flask --incluir "flask.py,flask/**/*.py,src/flask/**/*.py"
```

BLOQUE 2 · EJEMPLO GUIADO 2 · F-02

Quince años de Flask en números

Versión	Fecha	Archivos	Líneas	Agregadas / eliminadas	Commits por mes
0.1	2010-04	1	663	—	—
0.5	2010-07	12	1.970	+1.970 / −1.516	125
1.0	2018-04	20	7.712	+2.336 / −1.160	27
2.0.0	2021-05	21	8.254	+2.727 / −2.794	14
2.3.0	2023-04	22	8.799	+658 / −1.216	15
3.1.0	2024-11	24	9.187	+743 / −396	9

Lo que se ve

Hasta la 1.0 crece casi doce veces; después, 19 % en seis años y medio.

La 2.0.0 elimina más de lo que agrega.

Desde 2012, entre 14 y 28 commits por mes.

Fuente: ejemplo_flask/evolucion_por_version.csv, 20 versiones con funcionalidad nueva. Las versiones atípicas se explican con CHANGES.rst en el recorrido resuelto.

BLOQUE 2 · EJEMPLO GUIADO 3 · F-03

En qué se va el esfuerzo de mantenimiento

Tipo	Commits de la muestra	Sobre los que tocan el producto
Fuera del producto	24 de 40	—
Preventivo	5	31 %
Correctivo	4	25 %
Adaptativo	4	25 %
Perfectivo	3	19 %

Dos hallazgos de método

60 % de la muestra no toca el producto: por eso se usa --solo-fuente.

La heurística por palabras clave acierta en **15 de 40**.

La clasificación se hace leyendo el diff.

Muestra aleatoria de 40 commits sin merges, semilla 1. Clasificación manual en ejemplo_flask/muestra_cambios_clasificada.csv.

BLOQUE 2 · EJEMPLO GUIADO 4 · F-03

Los casos difíciles

Commit	La heurística dice	Leyendo el diff	Por qué
Added template_global, fixes #657	correctivo	perfectivo	Método nuevo con pruebas; el issue pedía la funcionalidad
remove deprecated RequestContext.g	adaptativo	preventivo	El entorno no cambió; se retira código propio obsoleto
add SESSION_COOKIE_PARTITIONED config	perfectivo	adaptativo	Los navegadores introdujeron las cookies particionadas
Pass maxsplit via kwarg to re.split	sin sugerencia	adaptativo	Python 3.13 marcó como obsoleto el argumento por posición

Un caso difícil bien razonado vale más en el informe que diez casos fáciles. Anoten por qué admitía dos tipos y cuál eligieron.

Qué leyes se observan en Flask

Ley	Evidencia	Contraevidencia	Veredicto
I · Cambio continuo	4 de 16 cambios al producto son adaptativos	—	Se observa
II · Complejidad	Menciones a «deprecat»: 27, 91 y 2 en 2.0.0, 2.2.0 y 3.0.0	type: ignore pasa de 58 a 105	Con matices
IV · Estabilidad	14 a 28 commits por mes desde 2012, con 13 a 209 autores	9 por mes en la 3.1.0	Se observa
V · Familiaridad	2.0.0 → 2.1.0, 2.2.0 → 2.3.0, 3.0.0 → 3.1.0: grande y luego pequeña	1.0 y 1.1.0 grandes seguidas	Se observa
VI · Crecimiento	De 663 a 9.187 líneas	Casi plano desde 2018	Con matices
VII · Calidad	—	—	Los datos no alcanzan

«Los datos no alcanzan» es una respuesta válida si se dice qué dato haría falta: aquí, la serie de defectos reportados por versión.

BLOQUE 2 · EJEMPLO GUIADO 6 · F-05 Y F-06

Etapa, deuda y recomendación

F-05 · Etapa: evolución

La 3.0.0 reorganizó las clases principales y la 3.2.0, en desarrollo, fusiona dos tipos de contexto: todavía hay cambios de estructura.

Señales de transición a servicio: actividad en descenso, tamaño casi constante y ninguna versión con funcionalidad nueva desde noviembre de 2024.

F-06 · Deuda vigente

Supresiones type: ignore (105 en 3.1.0, 90 hoy).

Alias y firmas obsoletas durante el período de la 3.2.0.

Despacho interno apoyado en objetos proxy globales.

Soporte de Python 3.9 y 3.10, ya retirado en la 3.2.0.

Recomendación: seguir con condiciones. Seguir los avisos de obsolescencia, fijar versiones y vigilar la salida de la 3.2.0.

El detalle de cada deuda, con origen, interés y pago, está en el recorrido resuelto.

EL HACER · TALLER EN SALAS

De la sesión salen con el proyecto medido

1	**Elegir el proyecto** Dos candidatos por grupo, revisados contra los seis criterios del F-01.	8 min
2	**Clonar con la historia completa** Sin --depth. Revisar git tag y decidir el patrón de versiones.	5 min
3	**Decidir qué se mide** Rutas del código fuente; qué queda fuera y por qué.	5 min
4	**Correr la herramienta** Con --solo-fuente y una semilla declarada.	5 min
5	**Clasificar los primeros cinco commits** Leyendo el diff, con la evidencia anotada.	7 min

Al salir de la sala: el F-01 en borrador y los dos CSV generados. El resto de la muestra se clasifica durante la semana.

PARA TENER AL LADO DURANTE EL TALLER

Comandos y criterios

Comandos

git clone URL del proyecto

git tag para ver las versiones

python herramientas/metricas_evolucion.py proyecto **--incluir** "rutas" **--solo-fuente --semilla 7**

git show commit para leer el diff

Criterios del proyecto (F-01)

Cinco años o más de historia pública

Ocho o más versiones con funcionalidad nueva

Registro de cambios y gestor de issues públicos

La herramienta termina en menos de diez minutos

Ningún otro grupo lo eligió

CIERRE · QUÉ SIGUE

De entender cómo cambia a controlar cómo cambia

ESTA SEMANA

Actividad 1.2

Clasificar la muestra completa, contrastar las leyes, ubicar la etapa y registrar la deuda. Verificar con el F-07 antes de entregar.

UNIDAD 2

Gestión de configuración

Qué elementos se ponen bajo control, cómo se identifican las versiones y cómo se decide qué cambio entra. Es lo que necesita la recomendación de fijar versiones.

El diagnóstico de esta unidad dice qué le pasa a un sistema con el tiempo. La gestión de configuración es la disciplina que permite controlarlo.

REFERENCIAS

Fuentes de la sesión

Bourque, P., & Fairley, R. E. (Eds.). (2014). Guide to the Software Engineering Body of Knowledge (SWEBOK v3.0). IEEE Computer Society.

Cunningham, W. (1992). The WyCash portfolio management system. En Addendum to the Proceedings of OOPSLA '92 (pp. 29–30). ACM.

Fowler, M. (2009). Technical debt quadrant. https://martinfowler.com/bliki/TechnicalDebtQuadrant.html

ISO/IEC/IEEE. (2022). ISO/IEC/IEEE 14764:2022 Software engineering — Software life cycle processes — Maintenance.

Lehman, M. M. (1980). Programs, life cycles, and laws of software evolution. Proceedings of the IEEE, 68(9), 1060–1076.

Lehman, M. M., Ramil, J. F., Wernick, P. D., Perry, D. E., & Turski, W. M. (1997). Metrics and laws of software evolution: The nineties view. En Proceedings of the Fourth International Software Metrics Symposium (pp. 20–32). IEEE.

Parnas, D. L. (1994). Software aging. En Proceedings of the 16th International Conference on Software Engineering (pp. 279–287). IEEE.

Rajlich, V. T., & Bennett, K. H. (2000). A staged model for the software life cycle. Computer, 33(7), 66–71.

Swanson, E. B. (1976). The dimensions of maintenance. En Proceedings of the 2nd International Conference on Software Engineering (pp. 492–497). IEEE.

Pallets. (2026). Flask [Repositorio de código fuente]. https://github.com/pallets/flask